

NOTTINGHAM TRENT UNIVERSITY
SCHOOL OF SCIENCE AND TECHNOLOGY

**TerraSight An AI-Powered Web Application for
Forecasting UK Regional Socioeconomic Indicators**

by

Bera Aksoy

in

2026

**Project report in part fulfilment
of the requirements for the degree of
Bachelor of Science with Honours**

In

Computer Science

I hereby declare that I am the sole author of this report. I authorize the Nottingham Trent University to lend this report to other institutions or individuals for the purpose of scholarly research.

I also authorize the Nottingham Trent University to reproduce this report by photocopying or by other means, in total or in part, at the request of other institutions or individuals for the purpose of scholarly research.

Signature

Bera Aksoy

ABSTRACT

This report talks about TerraSight, a web app that uses AI to predict five socioeconomic indicators in nine English regions from 2025 to 2035. Regional socioeconomic data in England – population, employment rate, average house price, rental price index, and housing completions – is currently spread across multiple government websites, providing only historical information with no forward-looking forecasts. TerraSight was developed to fill this gap.

The data science side of the project was based on the CRISP-DM methodology. The first step was collecting raw CSV data from five different UK government sources, which were ONS, DLUHC, HM Land Registry, VOA, and NOMIS. This data then had to be cleaned, standardised, and merged into a single dataset. The final dataset ended up with 171 rows and covers the years 2005 to 2023. Getting all of this data to work together was probably the hardest part of the whole project, mainly because each source had its own way of naming things. The column names were different, and more importantly the region names did not match at all. To give an example, one dataset would call a region "Eastern" while another called the same place "East of England". There were 27 of these naming differences in total, and they all had to be mapped down to 9 standard names before anything could be merged. Once the data was clean, lag variables and growth rates were added through feature engineering so that the models could pick up on trends from previous years.

Three models were then trained and compared: Linear Regression, Random Forest, and Gradient Boosting. Hyperparameters were tuned using GridSearchCV with 3-fold cross-validation, which resulted in 215 model fits across the five indicators. To avoid data leakage, a chronological split was used where data up to 2019 was used for training and 2020 to 2023 was kept for testing. Out of the three, Linear Regression gave the best results with an average MAPE of 1.81% and an R-squared of 0.9856, which is well below the project's target of 8% MAPE. The other two models, Random Forest and Gradient Boosting, struggled with the small dataset and showed signs of overfitting. In fact, both of them produced negative R-squared values on the rental price index, which means they actually performed worse than just predicting the average.

The trained model is served through a REST API built with FastAPI on the back end. Since the model can't directly predict a far-off year like 2030 from the most recent data in 2023, it works by going one year at a time. First, it predicts 2024.

Then, it updates the lag values with those predictions and uses them to predict 2025. This continues until it gets to the target year. For the frontend, a dashboard was built using React 19 and TypeScript with Vite as the build tool. It has four main pages: a home dashboard, a prediction page, a page for comparing two regions side by side, and an analytics page with more detailed technical data. Users can get forecasts for a single region, compare two regions against each other, or look at things like outlier detection and correlation analysis. The whole application is deployed publicly using Render for the backend and Vercel for the frontend. In terms of speed, the API response time was between 0.17 and 0.22 seconds, which is well below the goal of one second. All five indicators met the accuracy target, with MAPE values going from 0.11% for employment rate up to 6.52% for housing completions. It's important to note that there are some limitations though, such as the confidence intervals being approximate rather than statistically exact, the model not including outside factors like interest rates, and the assumption that past trends will continue to follow a straight line. These are discussed further in the report.

ACKNOWLEDGEMENTS

I want to thank my project supervisor, Arif Rahman, from the bottom of my heart for all the help, advice, and kindness he has shown me throughout this project. His helpful comments and support at every stage helped me get through tough times and shaped the direction of this work. I'm very thankful that he took the time and made the effort to help me grow as a student.

I also want to thank Nottingham Trent University for making this project possible by giving us the tools and space to learn.

TABLE OF CONTENTS

ABSTRACT	II
ACKNOWLEDGEMENTS	IV
TABLE OF CONTENTS.....	V
LIST OF FIGURES	VIII
LIST OF TABLES	IX
CHAPTER 1	1
INTRODUCTION	1
1.1 Introduction	1
1.2 Aims and Objectives.....	2
1.3 Scope	3
1.4 Benefits	4
1.5 Contribution and Novelty	4
1.6 Report Structure.....	5
CHAPTER 2	6
CONTEXT	6
2.1 Introduction	6
2.2 Regional Inequality In England.....	6
2.3 UK Open Government Data	7
2.4 Machine Learning For Socioeconomic Forecasting.....	7
2.5 Feature Engineering for Time Series.....	8
2.6 Evaluation Metrics	9
2.7 Existing Tools and Platforms.....	10
2.8 Technology Stack Selection.....	11
2.9 Summary	12
CHAPTER 3	13

NEW IDEAS	13
3.1 Introduction	13
The previous chapter looked at regional inequality in England, the gaps in the tools that are currently available, and why machine learning is a good fit for this kind of prediction. This chapter explains how TerraSight was actually designed based on those findings. It covers the development methodology, the functional and non-functional requirements, the system architecture, and the designs for both the data pipeline and the machine learning pipeline.13	
3.2 Development Methodology.....	13
3.2.1 Introduction	13
3.2.2 Business Understanding	14
3.2.3 Data Understanding	14
3.2.4 Data Preparation	14
3.2.5 Modelling	15
3.2.6 Evaluation	15
3.2.7 Deployment.....	16
3.2.8 Summary	16
3.3 Functional Requirements	17
3.4 Non-Functional Requirements.....	19
3.5 System Architecture.....	20
3.6 Data Pipeline Design	22
3.7 Machine Learning Pipeline Design	24
3.8 Summary	25
CHAPTER 4	26
IMPLEMENTATION OR INVESTIGATION	26
4.1 Introduction	26
The previous chapter presented the design, requirements, and pipeline architectures for TerraSight. This chapter goes through how all of that was actually built, in the order it happened. From project planning to data collection and building the pipeline, from exploratory analysis to model training, from the backend API to the frontend dashboard, and finally deployment and testing — every stage of the development process is covered here. The total codebase ended up at 4,112 lines, with 1,346 on the Python side and 2,766 on the frontend.26	
4.2 Project Planning	26
4.3 Data Collection and Pipeline Implementation.....	27
4.4 Exploratory Data Analysis	28
4.5 Model Training and Selection	28
4.6 Backend API Development.....	29

4.7 Frontend Development.....	30
4.8 Deployment and Testing	31
4.9 Summary	32
CHAPTER 5	33
RESULTS / DISCUSSION	33
5.1 Introduction	33
5.2 Model Performance Analysis	33
5.3 Api and System Performance	35
5.4 Objectives Evaluation.....	35
5.5 Requirements Evaluation	36
5.6 Limitations and Honest Assesment.....	37
5.7 Summary	38
CHAPTER 6	39
CONCLUSIONS / FUTURE WORK.....	39
6.1 Introduction	39
6.2 Conclusions	39
6.3 Future work.....	40
6.4 Legal, Social, Ethical and Professional Issues	41
6.5 Synoptic Reflections.....	42
REFERENCES	43
BIBLIOGRAPHY	45
APPENDIX A	46

LIST OF FIGURES

Figure 1	11
Figure 2	17
Figure 3	18
Figure 4	21
Figure 5	22
Figure 6	24
Figure 7	34
Figure 8	35

CHAPTER 1

INTRODUCTION

1.1 Introduction

There are big differences between regions in the UK compared to other major economies (McCann, 2020), as shown by the numbers. In 2023, the average price of a house in London was about £558,000. In the North East, that price drops to about £150,000. On top of this, anyone who wants to look into the regional socioeconomic situation in England currently has to visit at least five different government websites. Population data is on ONS, house prices are on HM Land Registry, housing completions are on DLUHC, the rental index is on VOA, and employment data is on NOMIS. None of these sources provide any forecasts or predictions about the future. The picture for employment rates is a bit different though the gap between regions went from 9.4 percentage points in 2005 down to 6.5 points in 2023. Understanding how these indicators will change over the next ten years is important for policymakers, city planners, and citizens.

TerraSight is an AI-supported web application that was developed to address the problems mentioned above. Currently, there is no freely available tool or website that brings all of this data together, provides future predictions, and allows comparisons between regions. This project was built specifically to fill that gap, and it does so in three stages. First, raw data from five different government sources is cleaned and combined into a single dataset. Second, machine learning models are trained on this data to generate predictions for the period 2025 to 2035. Finally, these predictions are presented through an interactive web dashboard that users can easily understand and work with. Through this application, users can look at socioeconomic indicators like population, employment rate, average house price, rental price index, and housing completions, analyse trends, or make comparisons between different regions.

Regional inequality in England has also become a key topic in politics in recent years. The government's "Levelling Up" policy (DLUHC, 2022) aims to close the socioeconomic gap between regions and underscores the importance of data-informed decision making in this area. However, while there are some tools that

provide access to regional data, none of them do everything that is needed. For example, ONS's Subnational Indicators Explorer can visualise regional data, platforms like Zoopla and Rightmove show house price information, and organisations like Cambridge Econometrics develop regional economic models. But none of these bring together five different indicators on a single platform while also providing future predictions using machine learning.

Being able to forecast regional trends is important when it comes to planning based on real evidence. This is the kind of data councils and governments turn to when they are thinking about the next ten or twenty years, especially for housing, jobs, and infrastructure. But the way these forecasting models are shared is a problem. They almost always come as static Excel files or PDF reports. These kinds of formats are not interactive, do not show data in real time, and you usually need statistical knowledge to read them properly. That is a big drawback.

In recent years, machine learning techniques have been used successfully in many different fields for making predictions and forecasts. Machine learning was picked for this project because it can predict more than one indicator at the same time and learn patterns automatically from data from different sources. Most traditional econometric models only look at one indicator at a time. A machine learning method, on the other hand, can look at all five indicators at the same time.

The difference in house prices is just one example of this inequality. Looking at the other indicators, similar gaps can be seen across the board. In terms of population, London has around 8.9 million people while the North East has the lowest population at just 2.7 million. Housing completions show a similar pattern in 2023, the South East completed 26,730 new homes, whereas in the North East that number was only 7,980. The rental price index, which uses 2015 as a base year of 100, has risen to as high as 128 in some regions. These differences show that regional inequality is not limited to just one indicator but is a multi-dimensional issue.

1.2 Aims and Objectives

The aim of this project is to address the problem of regional socioeconomic data in England being scattered across different sources, none of which offer any future predictions. To achieve this, the TerraSight application was developed, which uses machine learning to generate forward-looking forecasts for five indicators and brings the results together in an interactive web interface. The objectives of this project are as follows:

1. Collect raw data from five different government sources, clean it, and create a consistent and unified dataset.
2. Carry out exploratory analysis on the dataset to identify regional patterns and correlations between indicators.
3. Train three different machine learning models with hyperparameter optimisation, select the one with the best performance, and aim for an average MAPE of below 8% as the success criterion.
4. Build a REST API using FastAPI that serves predictions with confidence intervals and plain-English explanations, with a target response time of under one second.
5. Develop a four-page interactive dashboard using React and TypeScript, consisting of a home page, a prediction page, a comparison page, and an analytics page.
6. Deploy the completed application publicly using Render and Vercel, and include a disclaimer on all prediction pages.

1.3 Scope

This project covers nine regions in England based on the ITL1 classification, five socioeconomic indicators, and a forecast period of 2025 to 2035. Three regression models were trained using historical data from the years 2005 to 2023, and the system was built as a full-stack web application.

Some areas were intentionally left out of scope. Scotland, Wales, and Northern Ireland were not included because their data is published by different organisations and follows different formats. Time-series models like ARIMA were not tried because the main focus of the project was on machine learning regression models. Deep learning models were also not used as they were not expected to be effective with a dataset of this size. External variables such as interest rates and inflation were not included either, since this data is published at different time intervals and adding it would have significantly increased the complexity of the data merging process.

1.4 Benefits

This project can benefit a number of different groups. Policymakers can use the population and employment forecasts when making decisions about regional investment and resource allocation. Housing completions and rental index predictions can also be useful for shaping future housing policies. Citizens and property researchers can compare trends like house prices and rents across different regions, and use these forecasts to get an idea of what the socioeconomic situation in their area might look like in the future. One of the main advantages for users is that all of this data is brought together in a single interface, so there is no need to visit five separate government websites. On top of this, every prediction comes with a plain-English explanation, such as "Average House Price in London is projected to grow by 3.5% annually between 2023 and 2030", which means that even users without any statistics background can benefit from the application. Users can also make comparisons between regions and use more detailed analysis tools like outlier detection and correlation analysis.

1.5 Contribution and Novelty

The novelty of this project lies in addressing a gap that none of the platforms reviewed in Chapter 2 are able to fill on their own. There are currently three different categories of tools that touch on regional socioeconomic data in the UK. Government platforms like the ONS Subnational Indicators Explorer offer free historical data but do not make any predictions, and only show what has already happened. Commercial property sites like Zoopla and Rightmove cover house prices well but stay limited to that one area and do not produce any forward-looking forecasts. Cambridge Econometrics charges money for the services and predictions it provides, so it is not a platform that is free or open in the way TerraSight is.

TerraSight gets ahead of these platforms by closing this gap and doing what none of the platforms discussed so far can do. The first thing it does is bring five socioeconomic indicators together (population, employment rate, average house price, rental index, and housing completions) into a single unified dataset that can be accessed through one interface. Second, it uses machine learning to produce forecasts for the years 2025 to 2035 and achieves an average MAPE of 1.81% on the test set. And third, it lets every user benefit from it without any cost or membership requirement at all.

Beyond just combining these features, the app also automatically generates plain-English insights for every prediction. After the model runs, the system produces a short piece of text that explains in everyday language what the prediction means and gives the user some background information about it. This means that even users who are not particularly comfortable with reading charts can still get something useful out of the app. None of the platforms TerraSight was compared with offer this kind of feature.

When all of this is brought together, TerraSight becomes the only platform that allows even a non-expert user to obtain a prediction, see how confident the model is, get information from the plain-English explanation of the result, and compare two regions side by side. All of it sits in a single web interface, is completely free of charge, and asks the user to create no account of any kind.

1.6 Report Structure

The rest of the report is organized as follows. Chapter 2 introduces the story behind the project and provides context for the setting. It analyzes regional inequality in England and considers existing forecasting tools and platforms, investigates how machine learning is applied to socioeconomic predictions and explains why certain technologies were selected. Chapter 3 then continues to the system design, which includes CRISP-DM development methodology, functional and non-functional requirements, system architecture, data and machine learning pipeline designs. Chapter 4 presents development in chronological order: project planning, data collection, pipeline building, exploratory data analysis, model training and selection, backend API development, frontend dashboard construction, and testing. Chapter 5 is where the project is evaluated. It goes through the model accuracy metrics, looks at how each indicator performed, checks the API response times, and also talks about where the project falls short. Chapter 6 then brings everything together with the conclusions, a discussion on Legal, Social, Ethical and Professional Issues, and some reflections on the skills picked up while working on this project.

CHAPTER 2

CONTEXT

2.1 Introduction

There are a few key questions that played a role in shaping the design of this project: What is the current state of regional inequality in England? How do existing tools present this data, and what are they missing? Is machine learning a suitable and practical approach for these kinds of predictions? And what metrics should be used to measure and evaluate success? This chapter looks into the relevant literature and existing solutions to find answers to these questions.

2.2 Regional Inequality In England

McCann (2020) looked at 28 different indicators across 30 OECD countries and found that the UK has some of the worst regional inequality in the developed world. This inequality should not be thought of as just a matter of income or productivity, because it clearly shows itself across many different dimensions such as house prices, employment rates, population distribution, and housing supply. This is not a problem that only exists in England either, but England's centralised governance structure and the fact that its economic policies have been so focused on London have made the problem even worse. The government tried to respond to this with the Levelling Up White Paper in 2022, which basically set out a plan to close the gap between different parts of the country (DLUHC, 2022). However, as Fransham et al. (2023) point out, it is debatable whether this policy has actually created any new resources and whether local institutions have the capacity and funding to put it into practice. Martin et al. (2022) also looked into this and they do think the Levelling Up agenda is one of the biggest regional policy efforts the UK has ever seen, but they question whether the money behind it is actually enough to make a real difference. What all of this makes clear is that without proper tools that can look ahead and predict where things are going, it is very difficult for anyone to make good decisions about regional policy.

2.3 UK Open Government Data

One of the key building blocks of this project, and probably the biggest reason it was even possible to do, is the UK government's open data policy. All the data used in this project is published under the UK Open Government Licence version 3.0, which means it can be freely used for both commercial and academic purposes without any restrictions (The National Archives, no date).

However, there is a big difference between being able to access open data and actually being able to use it. The five sources used in this project show this very clearly. ONS (no date) publishes population figures on a yearly basis, but when it comes to employment data, NOMIS (no date) provides quarterly statistics from the Labour Force Survey. HM Land Registry (no date) reports house prices on a monthly basis through the UK House Price Index. ONS and VOA jointly publish a rental price index that uses 2015 as a base year of 100. DLUHC (no date) publishes housing completion figures based on the financial year, which runs from April to March. To be able to combine all five of these sources and actually use them together, everything had to be brought down to a yearly level first.

On top of the time intervals, there was also the problem of column names not being consistent. The population value for example could be called "population" in one dataset, but then the next dataset might call it "mid_year_estimate" or even just "all_ages", so the pipeline had to check for all of these and pick the right one. The region names were even worse. A place like the East of England might show up as "Eastern" in the DLUHC data or just "East" in another file. There were 27 different versions like this that all had to be fixed before the data could be merged. This is what the data pipeline was built for, and more details about how it works can be found in Chapter 4.

2.4 Machine Learning For Socioeconomic Forecasting

One of the first things that had to be figured out for this project was whether machine learning would even work well for this kind of forecasting. This is not as obvious as it might sound. Makridakis, Spiliotis and Assimakopoulos (2018) ran a pretty large study where they took 1,045 monthly time series from the M3 forecasting competition and tested popular machine learning methods against eight traditional statistical methods. What they found was that the machine learning models did not do as well, they were less accurate across the board and also took a lot more computing power to run. That said, their study was focused

on univariate time series, which is not really what this project does. This project does not try to predict one time series going forward year by year. What it does is take all 9 regions, line them up as rows in a table, and run regression across them. So the model is learning from all the regions at once, not just one.

Another thing worth mentioning is the data. The data was also a problem. 171 rows. That is all there is. Deep learning needs way more data than that, we are talking thousands of rows minimum, so it was not even considered. Random Forest and Gradient Boosting seemed like they could work since they are not as heavy, but they ended up overfitting. The rental price index was the worst case, both models got negative R-squared values on it. To put that simply, they were giving predictions that were less accurate than just using the average. So after testing everything, Linear Regression was the clear winner with the lowest MAPE across all five indicators.

2.5 Feature Engineering for Time Series

Models like ARIMA or Prophet are built to work with one variable at a time. So for London's population, you would need one model, and for London's house prices, another one. Multiply that across five indicators and nine regions and you are looking at 45 models, which is way too many. The easier option was to just put all the data into one big table and train a single regression model on it. Each row is just one region in one year. Regression does not know what year means though. 2015, 2023, it is all the same to it. London 2020 could be any row as far as the model knows. It has no way to look back at 2019 or 2018 by itself, that information has to be added manually into the row.

To deal with this, lag variables were added for each indicator. `population_lag1` contains the previous year's population, and `population_lag2` holds the population from two years before. Since the project does not just predict population, the same thing was done for employment rate, house prices, rental index, and housing completions as well, which created 10 lag variables in total. On top of that, growth rates showing the year-on-year percentage change were also calculated for each indicator, adding another 5 features. With all of this, the model can now look at not just the current year's values but also what happened in previous years and which direction the trends were heading. Together with the year and region columns, this gave the model 17 input features. Since the region name is categorical, it was converted into 9 numerical variables using

OneHotEncoder, and the numerical features were scaled using StandardScaler. In the end, the model was trained on a total of 25 features.

2.6 Evaluation Metrics

There is no single number that can tell you everything about how good a model is. That is why three metrics were used in this project, and they each measure something different.

MAPE shows the error as a percentage. For example, a MAPE of 2% means the predictions are off by about 2% from the actual values on average. The big advantage of MAPE in this project is that it does not depend on scale. That means an indicator like population, which is in the millions, can be compared with something like employment rate, which is a small percentage, using the same metric. Because of this advantage, MAPE was chosen as the main metric for the project. However, Hyndman and Koehler (2006) pointed out an important weakness of MAPE: if the actual value is close to zero, MAPE can become very high or even undefined. This is not a problem in this project though, since none of the five indicators take values anywhere near zero.

RMSE works differently. Because it squares the errors, it punishes large mistakes more heavily. But RMSE depends on the scale of the data, and this is an important limitation. For example, the RMSE for house prices in this project came out at £8,290, while for employment rate it was just 0.097 percentage points. Putting these two numbers next to each other and trying to compare them does not make any sense because they are on completely different scales. That is why RMSE was used for looking at individual indicators on their own, while MAPE was used for making comparisons across different indicators.

R-squared shows how much of the variance in the data the model can explain. A value of 1.0 means a perfect fit, while 0 means the model has not learnt anything at all. A negative R-squared means the model is actually doing worse than just predicting the average. In this project, the fact that Random Forest and Gradient Boosting both produced negative R-squared values on the rental price index means exactly that — for that particular indicator, those two models were giving worse predictions than if they had just used the average every time.

For this project, the goal was to keep the average MAPE under 8%. Going lower than 5% would not have been realistic with only 171 rows of data, and going above 10% would make the predictions pretty much pointless. 8% felt like the right middle ground.

2.7 Existing Tools and Platforms

When someone interested in regional socioeconomic data starts looking around, they will quickly notice that while historical data is available, there are either no future predictions at all, or they are locked behind paid services.

The ONS Subnational Indicators Explorer is the biggest and most comprehensive government platform in this area. It covers more than 38 indicators at the local authority level and gets updated every quarter. But it only shows historical data, there is no forecasting feature at all. It also presents each indicator separately and does not bring different indicators together in one view or show the relationships between them.

Zoopla and Rightmove are commercial platforms that focus on the housing market. Both of them offer quite detailed information on house prices and rental data, but that is where their coverage stops. Things like population, employment rate, or housing completions are outside of what they cover, and neither of them offers any kind of machine learning based predictions for the future.

Cambridge Econometrics is probably the nearest thing to what TerraSight tries to do. Their Local Economy Forecasting Model lets them put out regional forecasts that go into the future. But it costs money. You have to pay for a subscription, and their Knowledge Base is not open to the public. Students and researchers who do not have that kind of money are left with nothing.

Looking at all of these options together, not a single one of them does what TerraSight does. Five indicators, machine learning, no cost. That gap is exactly why this project exists.

Comparison of TerraSight with UK Regional Data Platforms

Feature	ONS	Zoopla / Rightmove	Cambridge Econometrics	TerraSight
Multiple socioeconomic indicators	Yes	No	Yes	Yes
Machine learning forecasting	No	No	Yes	Yes
Free / no cost	Yes	Yes	No	Yes
Side-by-side region comparison	No	No	No	Yes
Plain-English insights	No	No	No	Yes
Confidence intervals shown to user	No	No	No	Yes
Live web-based interface	Yes	Yes	No	Yes

Figure 1

2.8 Technology Stack Selection

There were a few options for each part of the tech stack, but here is why these ones were chosen. The backend runs on Python. That decision pretty much made itself because pandas (McKinney, 2010), NumPy (Harris et al., 2020), and scikit-learn, which are the three libraries the project depends on most, are all Python. What is nice about scikit-learn is that Linear Regression, Random Forest, Gradient Boosting, GridSearchCV, Pipeline, and StandardScaler are all in one place, so there was no need to install a bunch of different packages (Pedregosa et al., 2011). The API needed a framework, and after trying Flask briefly, FastAPI (Ramírez, no date) was the one that stuck. It does async out of the box, works nicely with type hints, and the Swagger page it generates on its own was a huge time saver when it came to testing endpoints.

React 19 (Meta Platforms, no date) is what the frontend runs on. The app has four pages and each one got its own file Home.tsx is the main dashboard, Predict.tsx does predictions, Compare.tsx handles comparisons, and Analytics.tsx shows the deeper stats. JavaScript would have worked fine but TypeScript (Microsoft, no date) was worth the extra effort. The API sends back a lot of JSON and without types it is easy to miss a wrong field name or a null value that breaks the page. Vite (VoidZero Inc., no date) does the builds. Any time something changes in the code, the browser picks it up instantly. Recharts

(Recharts Group, no date) handles all the graphs, and since it is React based, the charts are just components like everything else on the page.

The app is live on Render (Render, no date) for the backend and Vercel (Vercel, no date) for the frontend. Neither of them costs anything on the free plan, and since this project should be open to everyone, that was a must.

2.9 Summary

This chapter has laid out why this project is needed, what data is available, why machine learning was chosen, and how success will be measured. The UK is one of the most regionally unequal countries in the industrialised world. The tools that are currently out there either only show historical data or charge money for access. Machine learning regression models can work well on this kind of cross-sectional data, but the size of the dataset plays a big role in which model ends up being the right choice. MAPE was the go-to metric for this project. Population is in the millions and employment rate is a percentage, but MAPE lets you compare both without worrying about that. No tool available today does all of this in one place without charging for it. The next chapter gets into how TerraSight was actually designed, what the requirements were, and what methodology was followed.

CHAPTER 3

NEW IDEAS

3.1 Introduction

The previous chapter looked at regional inequality in England, the gaps in the tools that are currently available, and why machine learning is a good fit for this kind of prediction. This chapter explains how TerraSight was actually designed based on those findings. It covers the development methodology, the functional and non-functional requirements, the system architecture, and the designs for both the data pipeline and the machine learning pipeline.

3.2 Development Methodology

3.2.1 Introduction

The methodology question came up at the start. Three options were on the table Waterfall, Agile, and CRISP-DM. Waterfall and Agile come from software engineering. They both assume you can lock in the requirements before any work begins. That works for software. It does not work for data.

Data is the part you do not know in advance. Five sources, five different formats. Region names that do not match. Columns that contain different things than their names suggest. None of this is visible until the files are actually opened up.

CRISP-DM was built for this kind of work (Chapman et al., 2000). Six phases — Business Understanding, Data Understanding, Data Preparation, Modelling, Evaluation, Deployment. The arrows between them point forwards and backwards. Earlier phases can be revisited when something breaks later on. Figure 4.

3.2.2 Business Understanding

The first phase, Business Understanding, was used to clarify why and for what reason the project even needs to exist. The main problem tackled in this phase was regional socioeconomic inequality in England, along with the fact that no platform out there does forward-looking forecasting at the level TerraSight does. Concrete numbers made the case for the inequality side – an average house price of £558,000 in London against £150,000 in the North East. The output of this phase was six project objectives (listed in Section 1.2), and the technical success criterion was set as staying under an 8% MAPE.

3.2.3 Data Understanding

Data Understanding came down to opening up the five government sources and working out what was inside each one. The four organisations behind the data were ONS, DLUHC, HM Land Registry, and NOMIS. Each file was different. Column names did not line up. Time intervals were not the same. And the regions were spelt differently almost everywhere.

The region names took more time to fix than anything else. ONS has 'East of England'. DLUHC went with 'Eastern'. Another file shortened it to just 'East'. 27 of these mismatches across the five files, all for 9 regions. Yorkshire was the worst one – 'Yorkshire and The Humber', 'Yorkshire & The Humber', 'Yorkshire' on its own, 'Yorks and Humber' somewhere else. Population was no better. 'population' in one file, 'mid_year_estimate' in the next, and one source just went with 'all_ages'. Three different labels for the exact same number.

So the project had to step back into Business Understanding. The cleaning was way bigger than what had been planned for, and the scope of the work needed updating to match. This is exactly the kind of thing CRISP-DM is built for.

3.2.4 Data Preparation

This part was the longest and toughest phase of the project. 435 lines of Python went into `data_pipeline.py` just for this stage. The work was split into seven steps. First, separate loader functions were written for each source – `load_population`, `load_employment`, `load_house_prices`, `load_rental_index`, and `load_housing_completions`. Then `clean_columns` standardised every column name. `standardise_region` mapped all 27 region variants down to 9 canonical names. Different publishing frequencies – monthly, quarterly, and annual – were

brought down to a single annual scale. After that, the five sources were merged with `pandas.merge`. `add_growth_rates` calculated year-on-year percentage change for each indicator. `add_lag_features` came last and dropped in the one-year and two-year previous values as extra columns. The output of all seven steps was `master_dataset.csv` — 171 rows × 13 columns.

3.2.5 Modelling

Three regression models went head to head in the Modelling phase. Linear Regression as the baseline, plus Random Forest and Gradient Boosting as the more complex options. All three had to go through the same preprocessing pipeline so the comparison stayed fair. Numbers got scaled with `StandardScaler`. The `region` column got split into 9 binary columns by `OneHotEncoder`. Linear Regression has nothing to tune, so it just trained on the spot. Random Forest needed work though. Two values for `n_estimators` (100 and 200) and three for `max_depth` (6, 10, and None) were lined up for testing. Gradient Boosting needed even more — two values each for `n_estimators` (150, 200), `learning_rate` (0.05, 0.1), and `max_depth` (3, 4), plus `subsample` fixed at 0.8. `GridSearchCV` from `scikit-learn` (Pedregosa et al., 2011) ran every combination with 3-fold cross-validation. Five indicators, three models, 215 fits in total.

3.2.6 Evaluation

Three metrics were used to score the models — MAPE, RMSE, and R-squared. The train/test split had to be chronological, not random. If a row from 2021 leaked into training and then 2020 was used for testing, the model would already know what comes next. So everything up to 2019 went into training, 2020 to 2023 was kept aside for testing.

Linear Regression was the clear winner. 1.81% average MAPE, miles under the 8% target. Random Forest and Gradient Boosting did not come close. Random Forest sat at 4.02% MAPE, Gradient Boosting at 3.27%. Both still passed the target, but on the rental price index they produced negative R-squared values. Random Forest at -0.54, Gradient Boosting at -0.37. A negative R-squared basically means the model is doing worse than just guessing the average.

These numbers sent the project back to Modelling. Different hyperparameter ranges were tried. The results barely shifted. That made one thing clear — 171 rows is not enough for the more complex models. Simpler ones do better when the dataset is small.

3.2.7 Deployment

In the final phase, Deployment, the system was put live. The trained Linear Regression model was saved as `model.pkl` using `joblib`, and its metadata training date, per-indicator scores, feature columns went into a separate `metadata.json` file. On the backend side, FastAPI was used and 13 REST endpoints were defined (for example `/predict`, `/compare`, `/regions`, `/indicators`, `/analytics/timeseries`, `/analytics/outliers`, `/analytics/correlation`). The backend is deployed on Render. The frontend was written in React 19 and TypeScript and is deployed on Vercel. Since the frontend and backend run on different domains, CORS had to be enabled on the backend. The system is live at `terrasight-iota.vercel.app`, open to all users without any signup or fee, so anyone can benefit from it

3.2.8 Summary

The loops these six phases went through clearly showed why CRISP-DM is the right choice for this kind of project, and why it was a really good call for this project too. If a linear methodology like Waterfall had been used instead, the major problem of 27 region inconsistencies found in Data Understanding could not have been fed back into Business Understanding. The same goes for the Evaluation phase when Random Forest and Gradient Boosting underperformed, going back into Modelling to try different hyperparameter settings would not have been possible either. In that situation, the iterative nature of CRISP-DM once again allowed every new problem the project ran into to be handled inside the methodology, and made the whole development process a lot more efficient.

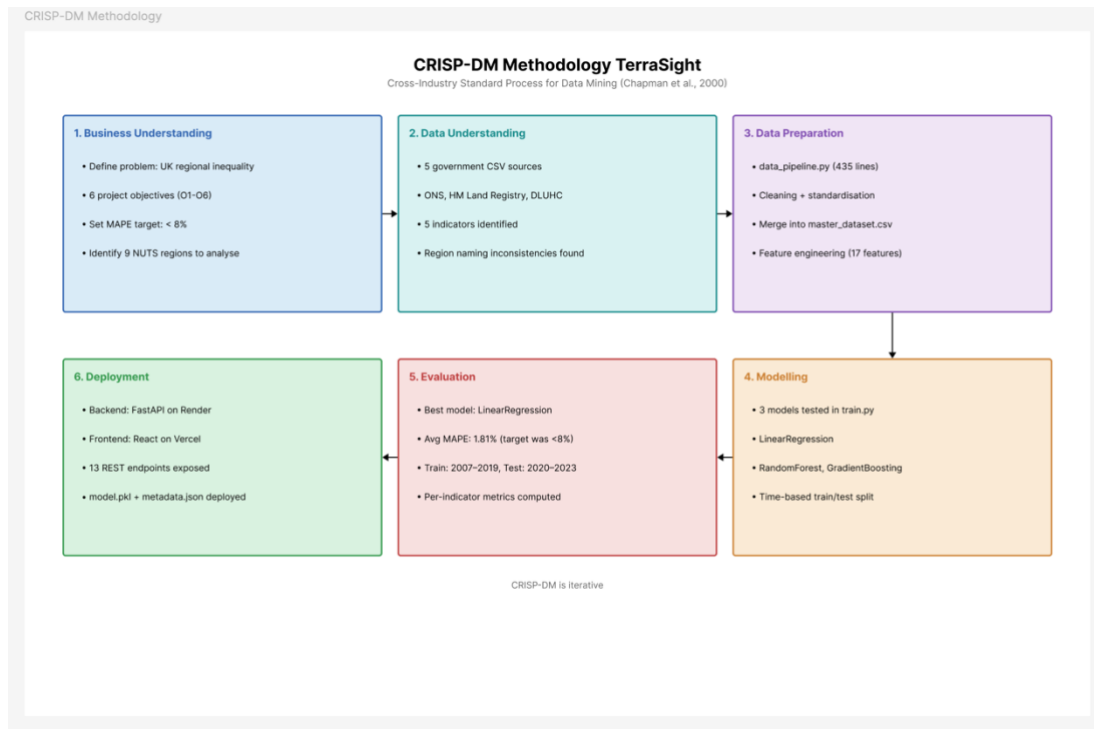


Figure 2

CRISP-DM Methodology

3.3 Functional Requirements

Functional requirements describe what users should be able to do with the system. These requirements were derived directly from the objectives set out in Chapter 1, and each one will be evaluated in Chapter 5 to determine whether it was met.

FR1: Prediction — the user should be able to select one of the nine regions and one of the five indicators, and get a prediction for any year between 2025 and 2035. This requirement is linked to Objectives 3 and 4.

FR2: Region comparison — the predictions for two different regions should be shown side by side on the same screen. This requirement is linked to Objective 5.

FR3: Confidence intervals — predictions should not just show a single number, they need to include an upper and lower bound as well so users can see how certain the forecast is. This requirement is linked to Objective 4.

FR4: Explanation text — most people are not going to look at a number like 9,312,000 and instantly know what it means for London's population. That is

why the app generates a line of text for each prediction, something along the lines of 'population in London is expected to grow by 1.2% per year'. No jargon, no tables, just a plain sentence that anyone can read. This requirement is linked to Objective 4.

FR5: Analytics tools — on top of predictions, the app should let users dig deeper into the data. That means being able to see trends over time, spot outliers, check moving averages, and look at how different indicators relate to each other. This requirement is linked to Objective 5.

FR6: Dashboard — the first thing a user sees when they land on the app should be a summary of all nine regions. They should not have to go searching through pages to get a general picture of what is going on. This requirement is linked to Objective 5.

FR7: Disclaimer — the app needs to be upfront about the fact that these predictions are not perfect. Every page with forecast data on it has to clearly say that the numbers are estimates and that nobody should be making financial decisions based on them. This requirement is linked to Objective 6.

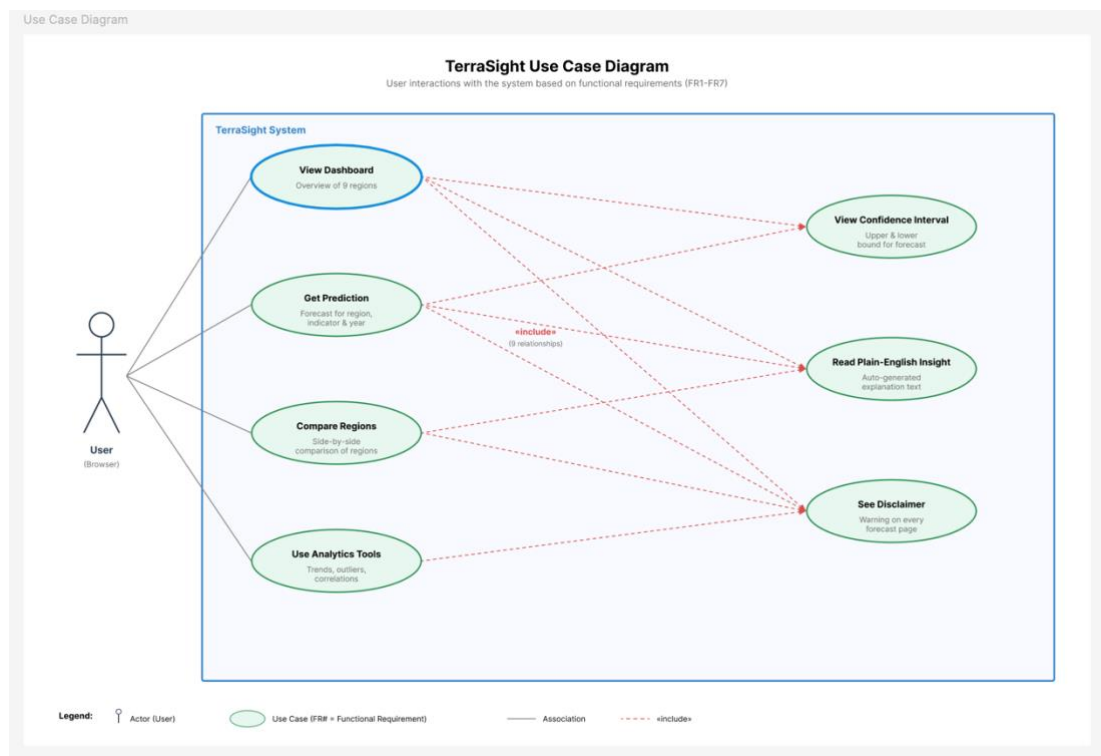


Figure 3

Use Case Diagram

3.4 Non-Functional Requirements

The previous section was about what the system needs to do. This section is about how well it needs to do it. Things like speed, accuracy, and whether the app is actually easy to use all fall under non-functional requirements.

NFR1: Performance — when someone clicks predict, they should see numbers on the screen within a second. Anything slower than that and it starts to feel like something is wrong. Long loading times would hurt the user experience and reflect badly on the project, so this target is critical. This requirement is linked to Objective 4.

NFR2: Accuracy — there is no point in showing predictions if they are nowhere near the real numbers. 8% average MAPE was the cutoff. Anything above that and users might as well flip a coin. This requirement is linked to Objective 3.

NFR3: Accessibility — all it takes is a browser and wifi. That is the only barrier to entry. You do not need to sign up or pay for anything, just go to the URL and start using it. This requirement is linked to Objective 6.

NFR4: Usability — give them the link, watch what they do. If they do not need to ask anyone for help, the job is done. No tutorials, no instructions needed. Buttons should be where you expect them, text should make sense on the first read, and nothing on the screen should make a user stop and wonder what it means. This requirement is linked to Objective 5.

NFR5: Reliability — no prediction is going to be spot on every time. That is why each one comes with two extra numbers, one higher and one lower, showing the range where the real value might actually land. On top of that, every page with a forecast on it has a disclaimer telling users not to take the numbers as gospel. This requirement is linked to Objectives 4 and 6.

NFR6: Compatibility — Chrome, Firefox, Safari, Edge. Safari user, Chrome user, Firefox user, they should all see the exact same thing. This requirement is linked to Objective 5.

3.5 System Architecture

The project has three pieces. Data, backend, frontend. They run separately and one does not need the other to function.

Data comes first. Five government sources, five CSV files. `data_pipeline.py` grabs them all, cleans up the mess, lines up the formats, and spits out one single dataset with 171 rows at the end. Once the data is ready, `train.py` runs the three models on it. The best one gets saved to a joblib file. Stuff like which indicators it works with and what MAPE it got are all written into `metadata.json`.

FastAPI powers the backend. There are five Python files in total and `main.py` is where all 13 endpoints are defined. `model.py` loads the trained model and handles prediction requests along with confidence interval calculations. `analytics.py` contains the functions for time series visualisation, outlier detection, and correlation analysis. `insights.py` generates a plain-English explanation for each prediction automatically. `config.py` holds configuration data like the list of regions and indicator details. The backend is hosted on Render.

The frontend was built with React 19 and TypeScript, compiled using Vite. There are four pages: `Home.tsx` gives a general overview as the main dashboard, `Predict.tsx` lets users get predictions for a single region, `Compare.tsx` allows two regions to be compared side by side, and `Analytics.tsx` provides more detailed analytical tools. All the charts are built using Recharts. Vercel takes care of hosting the frontend.

The frontend talks to the backend through the REST API. A user clicks something, the frontend fires off a fetch request, the backend does its thing and sends back JSON, and then the frontend takes that JSON and draws charts with Recharts. Since the frontend and backend live on different domains, CORS had to be turned on so they could actually communicate.

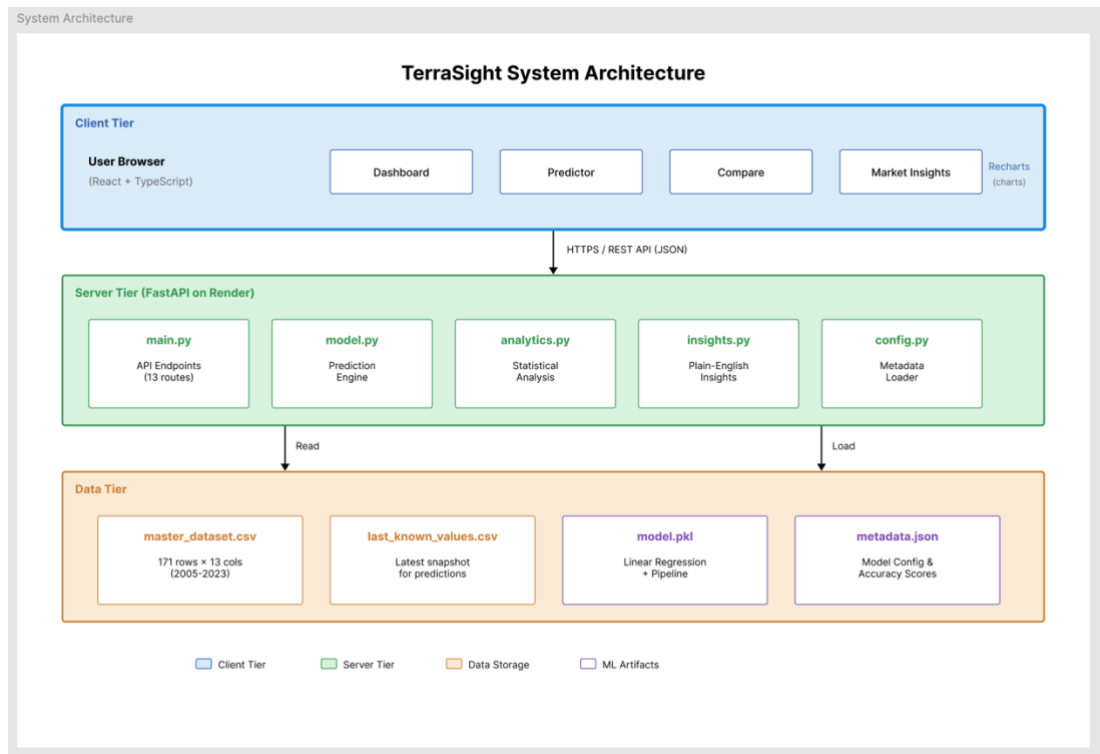


Figure 4

TerraSight System Architecture

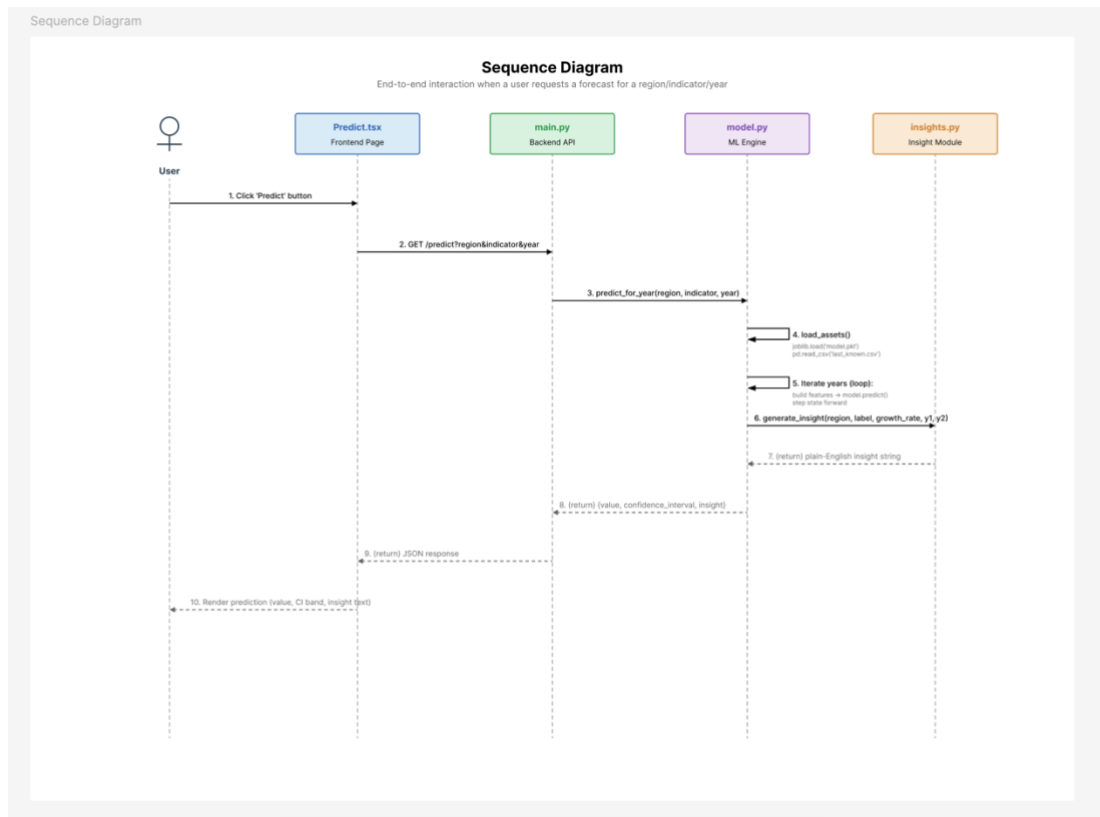


Figure 5

Sequence Diagram

3.6 Data Pipeline Design

Cleaning data should not take 436 lines of Python. But it did. The five CSV files were so different from each other that getting them into one usable table was a seven step process.

First thing that happens is loading. There is a separate function for each source — load_population, load_employment, load_house_prices, load_rental_index, load_housing_completions. One generic loader was not going to cut it because the files have nothing in common when it comes to structure.

The column names did not match at all. 'population' in one file, 'mid_year_estimate' in the next, and one source just went with 'all_ages'. Three different labels for the exact same number. clean_columns was written to go through each file and force all the names to match.

Region names were even worse and probably took more time to fix than anything else. ONS has 'East of England', DLUHC went with 'Eastern', and another file shortened it to just 'East'. 27 of these mismatches across the five files, all for 9 regions. `standardise_region` catches every one of them. Without it pandas would see 'Eastern' and 'East of England' and think they are two different regions, which would wreck the merge.

Publishing frequency was different for every source. House prices are monthly. Employment is every three months. Completions use the financial year, April through March. Merging these together only works if they are all annual, so the monthly data got averaged down, the quarterly data got combined, and the financial year data got assigned to the calendar year it started in.

With all five files finally using the same regions, same columns, and same time scale, pandas could merge them on region and year into one table. In step six, `add_growth_rates` calculates the year-on-year percentage change for each indicator. `add_lag_features` comes last and drops in the previous one and two year values as extra columns. After all seven steps, `run_pipeline` ties it together and what comes out the other side is `master_dataset.csv`.

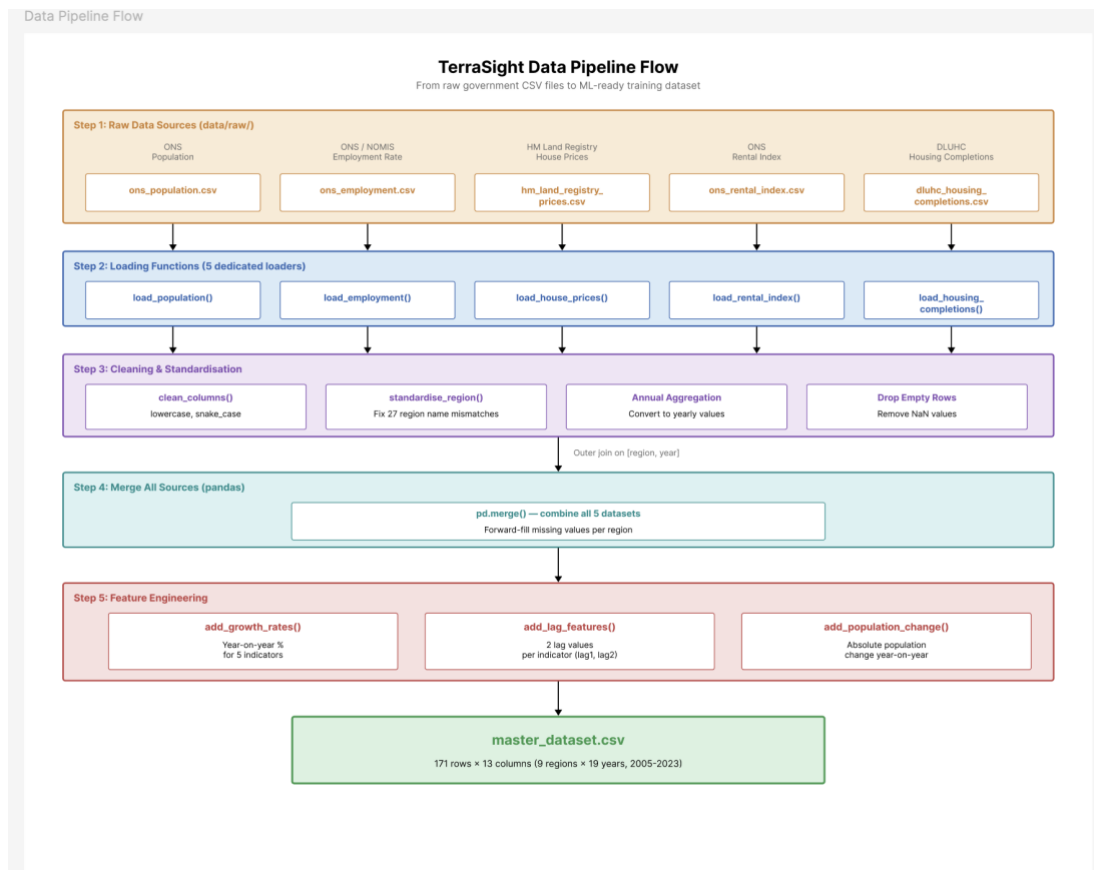


Figure 6

TerraSight Data Pipeline

3.7 Machine Learning Pipeline Design

Once the dataset is in good shape, `train.py` is what runs next. 355 lines of Python, all dedicated to one question which of the three models should the app actually use?

`master_dataset.csv` gets read in and then split by year. Anything from 2019 or before goes to training. 2020, 2021, 2022, 2023 those four years are only used for testing. The cut is based on time, not random. If a row from 2021 leaked into the training data and then 2020 got used for testing, the model would already know what comes next. That would make the results meaningless.

`build_preprocessor` handles the setup that all three models go through before training. Numbers get scaled with `StandardScaler`, all 16 columns. The region column gets split into 9 binary columns by `OneHotEncoder`. Linear Regression has no hyperparameters so it just trained straight away. Random Forest and

Gradient Boosting did though, so `get_candidate_models` defined the parameter grids and `GridSearchCV` tested every combination with 3-fold cross-validation. 215 model fits came out of that.

Once training is done, each model gets scored against the test data using `mape` and `rmse`. Every indicator is scored on its own, so there are five MAPE values and five RMSE values per model. Whichever model has the lowest MAPE on average is the winner. It gets dumped to a joblib file and everything about its scores, indicators, settings goes into `metadata.json`.

Hit the predict button and `model.py` wakes up. It grabs the model out of the joblib file and `_step_state_forward` handles the rest from there. Say a user wants 2030 the function starts by predicting 2024, takes those numbers and slots them in as the lag values, then predicts 2025 using those, and so on until it gets to 2030. `CONFIDENCE_PCT` is what gives each prediction its upper and lower bound.

3.8 Summary

This chapter covered how TerraSight was designed from every angle. It explained why CRISP-DM was chosen and how the project mapped onto its six phases. Seven functional and six non-functional requirements were defined, each one tied back to the project objectives. The system was designed as a three-layer architecture — data, backend, and frontend. Five sources go into the data pipeline, seven steps later there is one clean dataset. The ML pipeline then trains three models and keeps whichever one scores best. Chapter 4 goes into the details of how all of this got built.

CHAPTER 4

IMPLEMENTATION OR INVESTIGATION

4.1 Introduction

The previous chapter presented the design, requirements, and pipeline architectures for TerraSight. This chapter goes through how all of that was actually built, in the order it happened. From project planning to data collection and building the pipeline, from exploratory analysis to model training, from the backend API to the frontend dashboard, and finally deployment and testing — every stage of the development process is covered here. The total codebase ended up at 4,112 lines, with 1,346 on the Python side and 2,766 on the frontend.

4.2 Project Planning

The first bit of work happened in November 2025. The last commit was pushed at the end of April 2026. The way everything was split up followed CRISP-DM.

The first phase was research and planning. The regional inequality problem in England was looked into, the existing tools were reviewed, and the technology choices were made. There was a fair bit of back and forth during this stage — for example, ARIMA was originally being considered for the machine learning side, but after getting feedback from the supervisor it was dropped in favour of regression models. The project objectives were finalised by the end of this phase.

Next up was the data itself. Five CSV files and not a single one of them matched. Same data, totally different column names — 'population' here, 'mid_year_estimate' there. Even the dates and region names were written in their own way. The only way to find this out was to open them up and go row by row. That is when it hit that there were 27 different ways the regions had been named. On top of that, every file needed a separate check to confirm it had the right years and the right regions in it.

After spending enough time with the raw files to know what needed fixing, coding could begin. `data_pipeline.py` was written first. There was no point touching the models until the data was sorted out. That done, `train.py` took over. Three models went in, GridSearchCV tuned them, and one came out the winner. The API came together in FastAPI once the model side was sorted, and React handled the frontend which was the last thing to be built.

Development was finished by the end of April 2026 and the application was deployed live on Render and Vercel.

4.3 Data Collection and Pipeline Implementation

Chapter 3 covered the design of the data pipeline. This section talks about the problems that showed up during development and what was done to deal with them.

The first job was getting the actual files. All five sources publish their data as CSVs so they were downloaded one by one. ONS population was the easiest — the only catch was that some years had more than one version because ONS revises their numbers. The latest revision was used every time. The house price files from HM Land Registry were monthly and quite large, and the region names in them did not match the ones ONS uses. Getting employment data from NOMIS required signing up for an account first. DLUHC's housing completion figures were reported by financial year rather than calendar year, so a decision had to be made about how to convert them.

Region names were the kind of problem that looks small until you actually try to fix it. Every file spelt them differently and there was no shortcut, each one had to be checked by hand. Most of the differences were obvious but a few were sneaky — 'Yorkshire and The Humber' in one place, 'Yorkshire and Humber' in another, just one missing word. 27 mismatches in total. They all went into `REGION_ALIASES`.

Once all five files were merged, the result was a single table — 13 columns, 171 rows. Five of those columns are the raw indicators, another five are growth rates, there is one for population change, and the last two are region and year. A quality check on the final dataset turned up 9 NaN values in `rental_index_growth_rate`. The reason is simple — the rental index starts at 2015 as its base year, so the very first year for each region has nothing before it

to compare against. No previous year means no growth rate. These NaN values were handled in `train.py` to make sure they did not affect the model training.

4.4 Exploratory Data Analysis

No models were trained yet at this point. The data came first. First thing was going through the data row by row. Any gaps? Any blanks? Any numbers that stick out? All of that had to be checked before moving on.

9.48 million people live in the South East. The North East does not even hit 3 million, sitting at 2.71. House prices make the divide even harder to ignore. £558,000 in London. £150,000 in the North East — not even close. On the jobs side, 79.3% of people in the South East are employed compared to 72.8% in the North East. That 6.5% gap affects a lot of lives.

What is really telling is how the gap has moved over the years. Back in 2005 a house in London was about £252,000. Fast forward to 2023 and that number hit £558,000. The North East started at £112,000 and only managed to reach £150,000 in the same time. London more than doubled. The North East added about 30%.

The correlations between indicators were also checked. Population and house prices had a positive correlation of 0.675, which makes sense because areas with more people tend to have higher demand and higher prices. Population and employment rate had a weaker link at 0.364. House prices and the rental index came in at 0.333.

Going through all of this made it a lot clearer which features were likely to pull their weight once training started. It also confirmed with real data that the regional inequality described in Chapter 1 is backed up by the numbers.

4.5 Model Training and Selection

With everything cleaned and merged, the next question was simple — which model works best? Three were tested. Here is what happened.

Linear Regression, Random Forest, and Gradient Boosting all got a shot. Each model was trained on every indicator individually, so that is five separate training runs per model. Linear Regression went straight in with no tuning

needed. The other two had hyperparameters that needed figuring out, so GridSearchCV was set loose with 3-fold cross-validation. Between all three models and all five indicators, 215 fits were run.

Linear Regression crushed it. 1.81% average MAPE, miles under the 8% target. R-squared hit 0.9856. Population was the easiest indicator to get right at just 0.16% MAPE. Housing completions came in at 6.52%, which was the weakest of the five but still under the target. That one was always going to be harder — how many houses get finished in a year depends on things like government policy and what is happening in the construction industry, none of which the model can see.

Random Forest and Gradient Boosting were disappointing. Both of them produced negative R-squared values on the rental price index — Random Forest got -0.54 and Gradient Boosting got -0.37. A negative R-squared means the model is doing worse than just predicting the average every time. The most likely reason for this is the size of the dataset. 171 rows is enough for Linear Regression but not really enough for more complex models like Random Forest and Gradient Boosting. They memorised the patterns in the training data and then failed to generalise on the test data. In other words, they overfitted.

The takeaway from all of this is that a more complex model does not always mean better results. The size of the dataset matters a lot when choosing a model. Linear Regression, despite being the simplest of the three, gave the most reliable results for this particular dataset.

4.6 Backend API Development

After the model training was done, there needed to be a way for users to actually reach the model. That is what the backend API is for. It was built with FastAPI and is made up of five Python files.

[main.py](#) is the entry point and has 13 endpoints defined in it. These include /predict where users get a forecast for a single region, /compare for putting two regions side by side, /regions and /indicators which return the available options, and the analytics endpoints — /analytics/timeseries, /analytics/outliers, /analytics/correlation, and /analytics/stats/regions.

[model.py](#) is where predictions get made. The joblib model gets pulled in at startup and after that two functions do all the work. predict_for_year is the big

one. Hand it a region, an indicator, and a target year and `_step_state_forward` starts from the most recent real data, predicts the next year, updates the lags, predicts again, and keeps going until it hits the year that was asked for. The result comes back with a confidence range based on `CONFIDENCE_PCT`. `compare_two_regions` works the same way, it just does it for two regions instead of one and returns both.

[analytics.py](#) feeds the Analytics page. Time series data, outlier detection, correlation between indicators, stats broken down by region — all of that comes from functions in this file.

[insights.py](#) is tiny, just 40 lines, but take it out and users would be staring at raw numbers with no idea what they mean. `generate_insight` and `generate_comparison_insight` read the prediction and put together a sentence that makes sense to a normal person.

[config.py](#) is where the region names, indicator lists, and the `REGION_ALIASES` dictionary all live. Anything the other files need to look up about how the data is organised, they find it here.

Since the frontend and backend run on different domains, CORS had to be enabled in [main.py](#). One bonus of using FastAPI is the `/docs` page. It builds a Swagger interface on its own without writing any extra code, and that was used constantly to test whether the endpoints were returning the right data.

4.7 Frontend Development

An API on its own is not much use to anyone who is not a developer. All they would see is raw JSON. That is why the frontend was built — React 19, TypeScript, Vite, 2,766 lines by the time it was finished.

The app has four pages. Two components are shared across all of them — `Sidebar.tsx` handles the menu on the left, `layout.tsx` sets up the basic page structure.

`Home.tsx` is the landing page, 514 lines. All nine regions show up as soon as it loads. There is a mix of `AreaChart`, `BarChart`, and `LineChart` on the page depending on what kind of data is being shown. Everything is visible the moment it loads, no clicks required.

Predict.tsx is 369 lines and probably the page that matters most. Region goes in, indicator goes in, the API gets called. Recharts draws the forecast as a line and shades the confidence range either side of it. What makes it more useful is the sentence from [insights.py](#) that appears below — instead of just looking at a graph, the user reads something like 'population in London is expected to grow by 1.2% per year'.

Compare.tsx keeps it simple, 257 lines. There are two dropdowns — one per region. Once both are selected the forecasts appear on one LineChart next to each other. No need to open two tabs or try to remember what the first region looked like.

Analytics.tsx was by far the biggest page to build, 549 lines. This one is for the people who want to go further than predictions. Users can track how an indicator has moved over time, smooth things out with a moving average, spot anything that does not belong, or see if two indicators rise and fall together. Population and house prices are a good example of that — they clearly move in the same direction, and this page makes it easy to see.

Most requests come back fast enough that you barely notice. When one takes longer there is a loading animation so the page does not look frozen. If the backend sends back an error the page handles it and lets the user know what happened.

4.8 Deployment and Testing

With the development finished, the app needed to go live and be tested in a real environment.

The backend was put on Render — FastAPI, Uvicorn, the joblib file, and metadata.json were all part of the same deployment. Vercel takes care of the frontend and after that every code change goes live automatically. Any new push to the code and the site updates on its own. The app is live at terrasight-iota.vercel.app.

Everything worked on the local machine but that needed confirming on the actual server. Once the app was live, real prediction requests were used to test it. Fastest response was 0.17 seconds, slowest was 0.22. NFR1 gave it a full second so it was finishing in about a fifth of the time allowed. MAPE on the deployed app? 1.81%, same number as training. The model did not lose

anything in the move to production. 8% was the bar and this was nowhere near it.

Then came browser testing. Chrome worked. Firefox worked. Safari, same thing. Edge, no issues either. Charts looked identical across all four. A couple of deliberately wrong requests were thrown in as well just to make sure the app does not silently fail when something goes wrong.

Swagger played a big part in testing too. FastAPI creates a /docs page on its own and that was where every single endpoint got tested manually. Region names, indicator values, edge cases — all of it went through /docs before anything was considered done.

4.9 Summary

Six months. That is how long it took to go from an empty folder to a live application. First commit was November 2025, last one was end of April 2026. Five government CSVs were cleaned and merged by a 436-line pipeline. Three models were trained on what came out of it. Linear Regression ended up being the best at 1.81% MAPE. On the backend side, FastAPI serves 13 endpoints. React handles the frontend with four pages. Render got the backend, Vercel got the frontend, and from there it was a matter of testing. Is the API quick? Yes. Did the model's accuracy survive deployment? It did. Same results on Chrome, Firefox, Safari, Edge? All good.

The next chapter looks at all of this in detail and answers one question — did it actually work?

CHAPTER 5

RESULTS / DISCUSSION

5.1 Introduction

The earlier chapters explained why TerraSight was needed, how it was designed, and how it was built. This chapter steps back and looks at what actually came out of all that work. Model performance is broken down indicator by indicator, API and system performance are reviewed, the six objectives from Chapter 1 and the requirements from Chapter 3 are checked one by one, and the limitations of the project are discussed.

5.2 Model Performance Analysis

Linear Regression came out with a 1.81% average MAPE and an R-squared of 0.9856. The 8% target was never under threat. That said, the average hides quite a bit. Some indicators were almost spot on, others gave the model a much harder time.

Employment rate and population both came out near perfect, 0.11% MAPE and 0.16% MAPE. That is not really surprising though. Employment rates barely change from year to year, and population follows patterns that are fairly easy to predict.

Rental index came third at 0.23% MAPE. Since the index started in 2015 it has gone up at a pretty consistent rate, which made it straightforward for the model to follow.

House prices were where the model started to sweat a bit, 2.04% MAPE. Every region has its own housing market and they do not behave the same way at all. London keeps running ahead of the rest of the country and it has been doing that for years. Trying to capture that with one model that also has to work for the North East is not easy. RMSE was £8,290 — sounds bad on its own but when

houses cost £150,000 to £558,000, being off by eight thousand is less than a 6% miss.

Housing completions was always going to be the toughest one, and it was — 6.52% MAPE. The number of homes that get finished each year is driven by things like council decisions, government spending, and whether builders are busy or not. The model has no way of knowing any of that. But 6.52% still squeezed under the 8% target, so even the worst result was a pass.

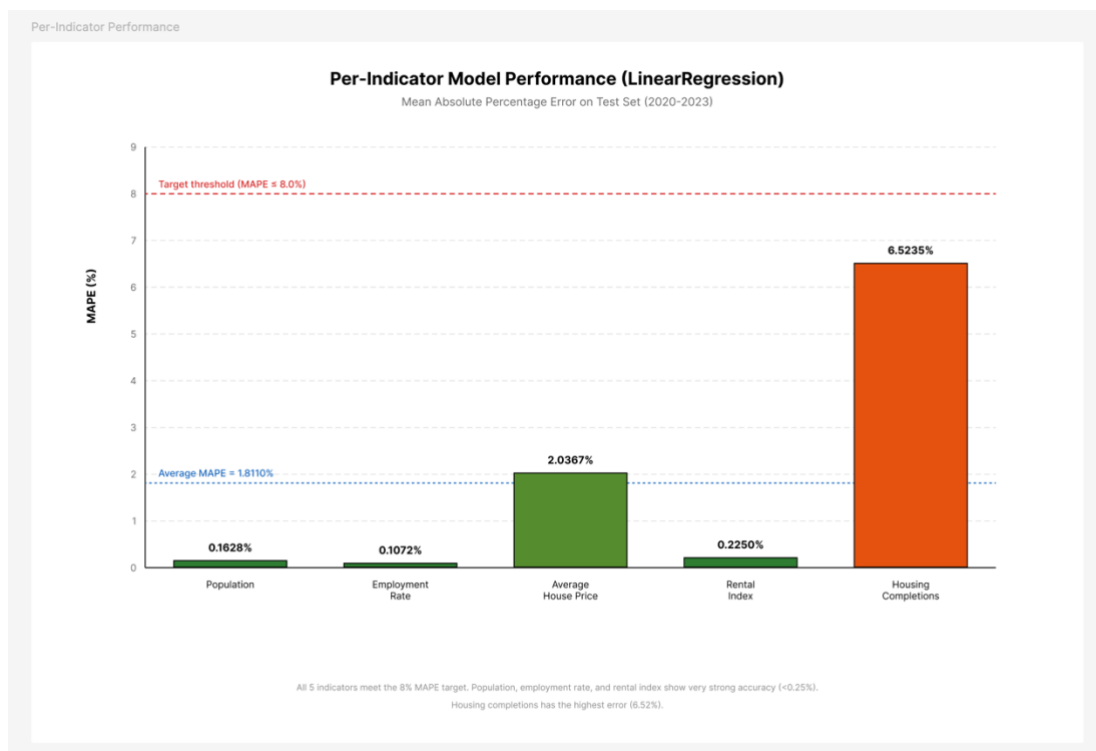


Figure 7

Random Forest and Gradient Boosting did not deliver. Both produced negative R-squared values on the rental index — Random Forest at -0.54 and Gradient Boosting at -0.37. This confirmed that the 171-row dataset was simply not big enough for these more complex models. They memorised the training data and could not generalise to the test set.

These results carry an important lesson. A more complex model does not automatically mean better predictions. The size of the dataset has to match the complexity of the model. In this project, Linear Regression gave the most reliable results.

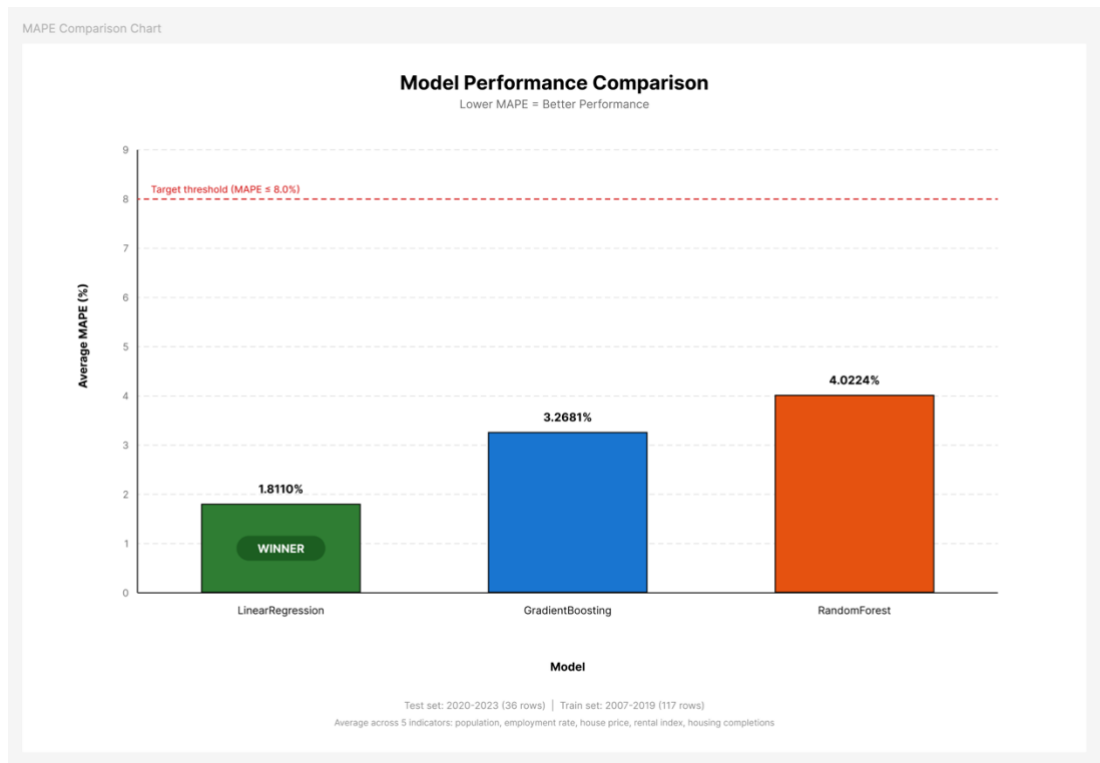


Figure 8

5.3 Api and System Performance

NFR1 set a one-second limit on API responses. Actual response times on the live app landed between 0.17 and 0.22 seconds. Nobody is going to notice a fifth of a second wait.

System reliability was also observed during testing. Render never went down during the whole project. Vercel picked up every code push and deployed it without any manual steps. As for accuracy, the 1.81% MAPE from training did not change once the model was running live. Same data in, same predictions out, regardless of whether it was running locally or on a server.

5.4 Objectives Evaluation

Chapter 1 laid out six things this project needed to achieve. This is what happened with each one.

First up — combine five government datasets into one. 436 lines of pipeline code later, `master_dataset.csv` existed. 171 rows, 13 columns, everything in one place. Done.

Second was exploratory analysis. The data needed looking at before any modelling could start. What came out of it was hard to ignore — the South East has 3.5 times the population of the North East, London house prices are nearly four times higher, and population and house prices showed a 0.675 correlation. The patterns were there. Done.

Third was training three models and making sure the best one stayed under 8% MAPE. Linear Regression hit 1.81% with an R-squared of 0.9856. The other two were tested but fell apart on the rental index, both going negative on R-squared. The target was 8%, the result was 1.81%. Done.

Fourth was building a REST API using FastAPI with confidence intervals, plain-English explanations for users who cannot read charts, and a response time under one second. The API ended up with 13 endpoints. Confidence intervals come from CONFIDENCE_PCT, and insights.py writes the plain-English explanations. As for speed, 0.17 to 0.22 seconds per request. Well under the one-second limit. Done.

Fifth was building a four-page React dashboard. Home, Predict, Compare, and Analytics were all developed, coming to 2,766 lines of frontend code in total. Done.

Sixth was deploying the app on Render and Vercel, making it free for everyone, and adding a disclaimer. The app is live at terrasight-iota.vercel.app, requires no signup or payment, and every prediction page shows a disclaimer message. Done.

Six objectives. Six passes.

5.5 Requirements Evaluation

The objectives were covered in 5.4. Now for the requirements. Chapter 3 listed thirteen of them. Did the app deliver on all of them?

Every requirement from Chapter 3 was checked after deployment. On the functional side, a user can open the app, pick any of the nine regions and five indicators, and get a prediction for any year between 2025 and 2035. That ticks

off FR1. Comparing two regions is just as straightforward — Compare.tsx draws both forecasts on one chart (FR2). Every prediction comes with a confidence range from CONFIDENCE_PCT and a plain-English sentence from insights.py, so FR3 and FR4 are both handled. The analytics page has trend lines, moving averages, outlier flags, and correlation checks, which is what FR5 asked for. The home page gives a full overview of all regions without clicking anything, that is FR6. And every page with a forecast on it has a disclaimer, covering FR7.

The non-functional side came down to six checks. Is it fast? 0.2 seconds per request, five times quicker than the one-second limit. Is it accurate? 1.81% MAPE against an 8% target. Free? Completely — no account, no charge. Usable? A first-time visitor can figure it out without reading anything. Transparent? Every forecast has a confidence range and a disclaimer. Does it work everywhere? Chrome, Firefox, Safari, Edge, all tested, all fine.

Thirteen requirements. Zero misses.

5.6 Limitations and Honest Assessment

Every objective was hit and every requirement was met, but that does not mean the project is without its weak spots. Being upfront about those matters more than pretending they do not exist.

171 rows. That is all the model had to work with. Nine regions, nineteen years. Linear Regression was fine with that but the other two needed more. Random Forest and Gradient Boosting both memorised the training data instead of learning real patterns, classic overfitting. More data would have helped — either going further back in time or breaking England down into smaller areas like counties instead of regions.

The project only covers England. Scotland, Wales, and Northern Ireland were left out because their data comes from different agencies and is published in different formats. Bringing them in would have made the pipeline significantly harder to build.

The model cannot see external events. Things like COVID-19, Brexit, or sudden changes in government policy all have a direct impact on the indicators, but none of that information is in the dataset. That is the main reason housing

completions came in as the weakest indicator at 6.52% MAPE how many homes get built is tied directly to government decisions and what is going on in the construction industry.

The nine NUTS1 regions are pretty broad. London for example is one region in this project, but anyone who lives there knows that property prices in Chelsea and property prices in Barking are worlds apart. That level of detail is lost at this scale.

There is also no way to know if the 2025 to 2035 predictions are any good yet. They cannot be checked until those years actually happen. Four years of test data is all the model has been measured against so far. Whether the longer-term forecasts hold up is something only time will answer.

5.7 Summary

This chapter went through every result the project produced. 1.81% MAPE from Linear Regression the 8% target was not even close to being a problem. The API came back in 0.17 to 0.22 seconds every time, which feels like nothing from the user's end. Every one of the six objectives passed. All thirteen requirements checked out. That said, the project is not perfect and this chapter did not try to hide that. 171 rows is not a lot of data, Scotland and Wales are missing, things like COVID and Brexit are nowhere in the model, and nobody will know if the 2035 predictions were right until 2035 actually comes. The final chapter pulls together the conclusions from this work and discusses what improvements and additions could be made in the future

CHAPTER 6

CONCLUSIONS / FUTURE WORK

6.1 Introduction

This report has covered why TerraSight was needed, how it was designed, how it was built, and what the results looked like. This final chapter brings it all together summarising the main conclusions, suggesting improvements for the future, going over the legal, social, ethical, and professional side of things, and reflecting on what was learnt along the way.

6.2 Conclusions

The goal when starting this project was to build a free web application that uses machine learning to forecast socioeconomic indicators across England's nine regions. That goal was reached.

Five government CSVs went through 436 lines of pipeline code and came out as one clean dataset 171 rows, ready for training. Linear Regression, Random Forest, and Gradient Boosting all had a go. Linear Regression won by a mile, 1.81% MAPE and 0.9856 R-squared. That is well under the 8% target. The API responds in 0.17 to 0.22 seconds and the four-page React dashboard gives users access to predictions, comparisons, and analytics tools. The app was deployed on Render and Vercel with no signup or payment required.

Comparing TerraSight to the tools discussed in Chapter 2 makes its position clear. The ONS Subnational Indicators Explorer has historical data but no forecasting. Zoopla and Rightmove only cover the housing market. Cambridge Econometrics can produce regional forecasts but charges for access. TerraSight is the only tool that brings five indicators together on one platform, runs machine learning predictions, and does it all without asking for a penny.

If there is one thing this project proved, it is that a fancier model does not mean a better result. Just like Makridakis et al. (2018) found in the M3 competition, more complex models do not always win. In this project, Random Forest and

Gradient Boosting both overfitted on 171 rows while Linear Regression, the simplest model of the three, delivered the most reliable results. That finding reinforces how much the size of the data matters when picking a model.

6.3 Future work

TerraSight works as it is, but there is plenty of room to make it better.

The first and probably most impactful change would be getting more data. 171 rows was enough for Linear Regression but the more complex models needed more to work with properly. There are two ways to do this — go further back in time past 2005 to add more years, or drop down from nine broad regions to local authority level. England has over 300 local authorities, which would push the dataset into the thousands of rows and give models like Random Forest and Gradient Boosting a real chance.

The model right now is blind to anything that is not in the dataset. COVID hit, house prices went weird, employment dropped — the model has no idea why. Adding external variables like pandemic data, Brexit timelines, or policy changes could help it understand those dips and spikes instead of just being confused by them. Housing completions would benefit the most from this since it is so tied to what the government decides to do.

Third, different types of models could be tried. This project only used regression models. With a bigger dataset, deep learning approaches like LSTM or time series models like Prophet would be worth testing. Ensemble methods that combine the strengths of different models could also be explored.

England is the only country covered at the moment. Scotland, Wales, and Northern Ireland would be the obvious next step, but each one has its own stats agency with its own way of publishing data. Pulling all of that into the same pipeline would take a lot of work.

Finally, a live data update feature could be added. The dataset is currently static and has to be updated by hand. If the ONS and other sources were accessed through their APIs, the data could be pulled in automatically and the model could be retrained on a regular schedule.

6.4 Legal, Social, Ethical and Professional Issues

Every software project carries legal, social, ethical, and professional responsibilities, and TerraSight is no different.

On the legal side, all the data used in this project is published under the UK government's Open Government Licence (OGL v3.0). This licence allows the data to be used, modified, and distributed for free. The project does not collect or process any personal data. No names, no emails, no identity information is asked from users, so there are no data protection obligations under GDPR. On top of that, the risk of predictions being misinterpreted was taken into account, which is why a disclaimer was added to every prediction page.

From a social perspective, TerraSight makes regional inequality more visible. Looking at the house price gap between London and the North East, where the difference is nearly four times, puts the scale of inequality into real numbers. The fact that the app is free and open to everyone without any registration means access to this kind of data has been democratised. Researchers, students, journalists, and policymakers can all explore regional trends without needing an expensive subscription.

There is an ethical side to this too. Not everyone will understand that a prediction is just a prediction. If someone pulls up London's 2030 forecast and treats that number like it is set in stone, they could end up making money decisions based on something that might not happen. That would be a problem. Every prediction page has a disclaimer making it clear that these are estimates, not promises, and that they should not be used for financial decisions. Confidence intervals are also shown so users can see how certain or uncertain each forecast is.

On the professional side, the project was developed in line with the BCS Code of Conduct (BCS, 2022). The code says you should give people accurate information, not lead them astray, and be honest about what your system can and cannot do. Chapter 5 did not hide the project's weak points, and every prediction page carries a disclaimer. That is what following those principles looks like in a real project.

6.5 Synoptic Reflections

This project showed me that I cannot just stop at what university teaches and that I always need to go beyond it. I learnt that things will not work on the first try and that instead of giving up I need to keep testing, keep failing, and keep pushing until I get it right. This project made me feel that more than anything.

The pipeline is where I learnt this the hard way. I opened the data and found 27 different ways of writing 9 region names. I just sat there. No idea what to do. I was convinced for a bit that the merge was just not going to happen. But giving up was not an option so I went through every file, row by row, writing down which names did not match. That is how REGION_ALIASES was born. If the data had been clean I probably would not know half of what I know now. The mess is what taught me. It was a project where I even learnt from my own mistakes.

Model selection was a similar story. I went in thinking the more complex models would give the best results. Negative R-squared was not something I was expecting to see. It caught me off guard. After looking into it I understood that 171 rows was simply not enough for those models, and that changed the way I think about picking a model.

Talking to the supervisor on a regular basis helped more than I expected. Dropping ARIMA was a decision that came out of one of those meetings and it was important for the project.

When I look back at who I was before this project, the change is not just technical. I learnt how to define a problem, research it, and solve it. On top of that, I noticed that my confidence in myself grew in a way I was not expecting. I trust myself a lot more now than I did before this project started.

REFERENCES

BCS (2022) *Code of conduct for BCS members*. Version 8. Swindon: BCS, The Chartered Institute for IT. Available at: <https://www.bcs.org/media/2211/bcs-code-of-conduct.pdf> (Accessed: 25 March 2026).

Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C. and Wirth, R. (2000) *CRISP-DM 1.0: Step-by-step data mining guide*. SPSS Inc. Available at: <https://www.kde.cs.uni-kassel.de/lehre/ws2012-13/kdd/files/CRISPWP-0800.pdf> (Accessed: 5 January 2026).

Department for Levelling Up, Housing and Communities (2022) *Levelling Up the United Kingdom*. White Paper, CP 604. London: HM Government. Available at: <https://www.gov.uk/government/publications/levelling-up-the-united-kingdom> (Accessed: 15 November 2025).

Department for Levelling Up, Housing and Communities (no date) *Live tables on housing supply: indicators of new supply*. Available at: <https://www.gov.uk/government/statistical-data-sets/live-tables-on-house-building> (Accessed: 26 November 2025).

Fransham, M., Herbertson, M., Pop, M., Bandeira Morais, M. and Lee, N. (2023) 'Level best? The levelling up agenda and UK regional inequality', *Regional Studies*, 57(11), pp. 2339–2352. doi: 10.1080/00343404.2022.2159356. Available at: <https://www.tandfonline.com/doi/full/10.1080/00343404.2022.2159356> (Accessed: 18 January 2026).

Harris, C.R., Millman, K.J., van der Walt, S.J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N.J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M.H., Brett, M., Haldane, A., Fernández del Río, J., Wiebe, M., Peterson, P., Gérard-Marchant, P., Sheppard, K., Reddy, T., Weckesser, W., Abbasi, H., Gohlke, C. and Oliphant, T.E. (2020) 'Array programming with NumPy', *Nature*, 585(7825), pp. 357–362. doi: 10.1038/s41586-020-2649-2. Available at: <https://www.nature.com/articles/s41586-020-2649-2> (Accessed: 8 February 2026).

HM Land Registry (no date) *UK House Price Index reports*. Available at: <https://www.gov.uk/government/collections/uk-house-price-index-reports> (Accessed: 25 November 2025).

Hyndman, R.J. and Koehler, A.B. (2006) 'Another look at measures of forecast accuracy', *International Journal of Forecasting*, 22(4), pp. 679–688. doi: 10.1016/j.ijforecast.2006.03.001. Available at: <https://www.sciencedirect.com/science/article/abs/pii/S0169207006000239> (Accessed: 12 January 2026).

Makridakis, S., Spiliotis, E. and Assimakopoulos, V. (2018) 'Statistical and machine learning forecasting methods: Concerns and ways forward', *PLoS ONE*, 13(3), e0194889. doi: 10.1371/journal.pone.0194889. Available at:

<https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0194889>

(Accessed: 30 January 2026).

Martin, R., Pike, A., Sunley, P., Tyler, P. and Gardiner, B. (2022) "'Levelling up' the UK: reinforcing the policy agenda", *Regional Studies, Regional Science*, 9(1), pp. 794–817. doi: 10.1080/21681376.2022.2150562. Available at:

<https://www.tandfonline.com/doi/full/10.1080/21681376.2022.2150562> (Accessed: 12 January 2026).

McCann, P. (2020) 'Perceptions of regional inequality and the geography of discontent: Insights from the UK', *Regional Studies*, 54(2), pp. 256–267. doi: 10.1080/00343404.2019.1619928. Available at:

<https://www.tandfonline.com/doi/full/10.1080/00343404.2019.1619928> (Accessed: 8 January 2026).

McKinney, W. (2010) 'Data structures for statistical computing in Python', in van der Walt, S.J. and Millman, K.J. (eds.) *Proceedings of the 9th Python in Science Conference*. Austin, TX, 28 June–3 July 2010, pp. 56–61. doi: 10.25080/Majora-92bf1922-00a. Available at: <https://proceedings.scipy.org/articles/Majora-92bf1922-00a> (Accessed: 5 February 2026).

NOMIS (no date) *Official census and labour market statistics*. Office for National Statistics. Available at: <https://www.nomisweb.co.uk/> (Accessed: 20 November 2025).

Office for National Statistics (no date) *Index of Private Housing Rental Prices, UK*. Available at: <https://www.ons.gov.uk/economy/inflationandpriceindices/bulletins/indexofprivatehousingrentalprices/previousReleases> (Accessed: 23 November 2025).

Office for National Statistics (no date) *Population estimates*. Available at: <https://www.ons.gov.uk/peoplepopulationandcommunity/populationandmigration/populationestimates> (Accessed: 22 November 2025).

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M. and Duchesnay, E. (2011) 'Scikit-learn: Machine learning in Python', *Journal of Machine Learning Research*, 12, pp. 2825–2830. Available at: <https://www.jmlr.org/papers/volume12/pedregosa11a/pedregosa11a.pdf> (Accessed: 18 February 2026).

The National Archives (no date) *Open Government Licence v3.0*. Available at: <https://www.nationalarchives.gov.uk/doc/open-government-licence/version/3/> (Accessed: 28 November 2025).

BIBLIOGRAPHY

Meta Platforms (no date) *React: The library for web and native user interfaces*. Available at: <https://react.dev/> (Accessed: 15 March 2026).

Microsoft (no date) *TypeScript documentation*. Available at: <https://www.typescriptlang.org/docs/> (Accessed: 15 March 2026).

Ramírez, S. (no date) *FastAPI: Modern, fast (high-performance), web framework for building APIs with Python*. Available at: <https://fastapi.tiangolo.com/> (Accessed: 12 March 2026).

Recharts Group (no date) *Recharts: Redefined chart library built with React and D3*. Available at: <https://recharts.github.io/> (Accessed: 16 March 2026).

Render (no date) *Render documentation*. Available at: <https://render.com/docs> (Accessed: 18 March 2026).

Vercel (no date) *Vercel documentation*. Available at: <https://vercel.com/docs> (Accessed: 18 March 2026).

VoidZero Inc. (no date) *Vite: Next Generation Frontend Tooling*. Available at: <https://vite.dev/> (Accessed: 16 March 2026).

APPENDIX A

Project Structure

```
|— data_pipeline.py      # Main data pipeline
|— train.py             # Model training script
|— requirements.txt      # Root-level Python dependencies
|— README.md            # Project overview and setup
|— data/
|   |— processed/
|   |   |— master_dataset.csv # Cleaned dataset (171 rows × 13 columns)
|   |   |— last_known_values.csv # Latest values used for prediction
|   |— models/
|   |   |— model.pkl          # Trained Linear Regression model
|   |— backend/
|   |   |— main.py            # FastAPI entry point (13 REST endpoints)
|   |   |— model.py           # Prediction engine
|   |   |— analytics.py       # Statistical analysis functions
|   |   |— insights.py        # Plain-English insight generator
|   |   |— config.py          # Configuration loader
|   |   |— metadata.json      # Model metadata and accuracy scores
|   |   |— model.pkl          # Trained model copy for backend
|   |   |— requirements.txt    # Backend Python dependencies
|   |— frontend/
|   |   |— package.json       # Frontend dependencies
|   |   |— vite.config.ts     # Vite build configuration
|   |   |— tsconfig.json      # TypeScript configuration
|   |   |— vercel.json        # Vercel deployment configuration
|   |— src/
|   |   |— App.tsx            # Root React component
|   |   |— main.tsx           # Application entry point
|   |   |— pages/
|   |   |   |— Home.tsx       # Dashboard page
|   |   |   |— Predict.tsx    # Prediction page
|   |   |   |— Compare.tsx    # Region comparison page
|   |   |   |— Analytics.tsx  # Analytics page
|   |   |— components/
|   |   |   |— Sidebar.tsx    # Navigation sidebar
|   |   |   |— layout.tsx     # Page layout wrapper
|   |   |— services/
|   |   |   |— baseapi.tsx    # API request wrapper
|   |   |   |— land_price.tsx # Indicator-specific API calls
|   |   |— hooks/
|   |   |   |— useStaggerReveal.ts # Animation hook
```

Requirements

Python 3.10 or higher
Node.js 18 or higher
npm 9 or higher

Backend dependencies (listed in backend/requirements.txt):

```
fastapi==0.135.1
uvicorn==0.42.0
pandas==3.0.1
numpy==2.4.3
scikit-learn==1.8.0
scipy==1.17.1
joblib==1.5.3
pydantic==2.12.5
```

Data pipeline and training dependencies (listed in requirements.txt):

```
pandas==3.0.1
numpy==2.4.3
scikit-learn==1.8.0
scipy==1.17.1
joblib==1.5.3
matplotlib==3.10.3
```

Frontend dependencies (listed in frontend/package.json):

```
react==19.2.4
react-dom==19.2.4
react-router-dom==7.13.1
typescript==5.9.3
vite==7.3.1
recharts==3.8.0
tailwindcss==4.2.1
axios==1.13.6
```

Installations

If you follow these steps, you can set up the app locally. The front end and back end work as different services that need to be started separately.

Backend Setup

```
cd backend
python -m venv venv
source venv/bin/activate      # macOS / Linux
# venv\Scripts\activate      # Windows
pip install -r requirements.txt
uvicorn main:app --reload
```

The backend will start on <http://localhost:8000>. The interactive Swagger documentation is available at <http://localhost:8000/docs>.

Frontend Setup

```
cd frontend
npm install
npm run dev
```

The front end will connect to the local backend quickly at <http://localhost:5173> when it starts up.

Dataset Setup

The project takes data from five UK government sources that is open to the public. The `master_dataset.csv` file in the `data/processed/` directory is the cleaned and merged dataset. It has 171 rows that cover the nine NUTS1 regions of England from 2005 to 2023.

The five raw data sources are:

- ONS Population Estimates (annual)
- NOMIS Labour Force Survey (employment rate, quarterly)
- HM Land Registry UK House Price Index (monthly)
- ONS Index of Private Housing Rental Prices (quarterly)
- DLUHC Live Tables on Housing Supply (financial year)

To regenerate the master dataset from raw sources, run

```
python data_pipeline.py
```

This executes the data pipeline (load, clean, standardise, merge, handle missing values, add growth rates and lag features) and writes the result to `data/processed/master_dataset.csv`.

How to Run the System

The full workflow consists of three steps

1-Generate the dataset

```
python data_pipeline.py
```

2-Train the model

```
python train.py
```

Linear Regression, Random Forest, and Gradient Boosting are trained on five indicators (215 fits in total via GridSearchCV with 3-fold cross-validation). The best model is then chosen and saved to `models/model.pkl` and `backend/metadata.json`.

3-Start the applications

```
# Terminal 1 - backend
cd backend && uvicorn main:app --reload
```

```
# Terminal 2 - frontend
cd frontend && npm run dev
```

Description of Each Source File

`data_pipeline.py`

The data pipeline. It has a separate loader function for each of the five sources (`load_population`, `load_employment`, `load_house_prices`, `load_rental_index`, `load_housing_completions`), a `clean_columns` function that lowercases and underscores column names, and `standardise_region` which uses the

REGION_ALIASES dictionary to map 27 different region naming variants down to the 9 canonical names. After loading and cleaning, frequencies are converted to annual, datasets are merged on region and year, and add_growth_rates and add_lag_features compute the engineered features. The output is master_dataset.csv with 171 rows and 13 columns.

train.py

Trains and evaluates three models on the cleaned dataset. Uses a chronological split: 2007–2019 for training, 2020–2023 for testing (chronological because shuffling time series would leak information). Preprocessing is done with StandardScaler for numeric columns and OneHotEncoder for the region column. Hyperparameters are tuned with GridSearchCV using 3-fold cross-validation, and models are scored on MAPE, RMSE and R-squared per indicator. The winner per indicator is saved to model.pkl with metadata in metadata.json.
backend/main.py

FastAPI entry point. Defines 13 REST endpoints: /predict and /compare for the core forecasts, /regions and /indicators for available options, /model/info for the model's accuracy scores, and four analytics endpoints (/analytics/timeseries, /analytics/outliers, /analytics/correlation, /analytics/stats/regions). CORS is enabled so the React frontend can call it.

backend/model.py

Loads the trained model on startup and handles prediction. The predict_for_year function uses an iterative step-forward approach: it starts from the most recent real data and walks one year at a time up to the target year, updating lag values at each step. compare_two_regions returns parallel forecasts for two regions. Confidence intervals come from the CONFIDENCE_PCT constant (currently 0.08, so plus or minus 8%).

backend/analytics.py

The statistical functions used by the Analytics page. Includes time series with rolling moving averages and year-on-year growth, outlier detection using the IQR method, Pearson correlation between indicators with the top correlated pairs ranked, and per-region stats (mean, median, std, min, max).

backend/insights.py

Generates the plain-English insight sentences shown next to each prediction. generate_insight produces a sentence describing the projected change for a single forecast, and generate_comparison_insight does the same for a two-region comparison. It uses simple string templates, no language model is involved.

backend/config.py

Loads the metadata.json file that train.py writes, with hardcoded fallback defaults if the JSON is missing for some reason. Exports META_DATA which holds the nine regions, five indicators with their display labels, the train and prediction year ranges, the seventeen feature columns, MAPE target, and model type. Also exports SIDEBAR_OPTIONS for the frontend navigation.

frontend/src/pages/Home.tsx

The landing page dashboard. Shows an overview of all nine regions across all five indicators using a mix of AreaChart, BarChart and LineChart components from Recharts.

frontend/src/pages/Predict.tsx

The prediction page. The user picks a region, an indicator and a target year, and the page shows the forecast as a line chart with a shaded confidence band along with the plain-English insight from the backend.

frontend/src/pages/Compare.tsx

The comparison page. Lets the user pick two regions and shows their forecasts side by side on a single chart.

frontend/src/pages/Analytics.tsx

The analytics page. Provides deeper analysis: trends over time, moving averages, outlier detection and correlations between indicators.

Outputs Generated

The system produces

master_dataset.csv (171 rows, 13 columns) - cleaned and merged dataset
last_known_values.csv - most recent values per region/indicator
model.pkl - trained Linear Regression model saved with joblib
metadata.json - per-indicator MAPE, RMSE and R-squared scores, training date, feature columns, model parameters
These end up in data/processed/, models/ and backend/ and are used directly by the deployed application.

Deployed Applications

Live Deployment <https://terrasight-iota.vercel.app>.